Distributed Systems Project Topics

System and Graph Projects

1. Dynamo-Inspired Fault-Tolerant Key-Value Store

Implement a Dynamo-inspired key-value store with consistent-hash partitioning and replica copies. Detect unavailable nodes, continue operations within a stated failure tolerance, and repair missing or stale replicas when nodes return. Demonstrate recovery without losing acknowledged writes under the chosen write policy. Use 4–6 processes, a GET/PUT API, heartbeat-based failure detection, and at least three copies per key. Store versions durably and compare replica digests to identify missing updates. Kill a replica during writes, restart it, and measure repair time and request availability. Concepts: consistent hashing, replication, failure detection, anti-entropy, durability.

2. Dynamo-Inspired Store with Configurable Consistency

Implement a replicated key-value store with configurable read and write quorums. Track concurrent versions and support an explicit conflict-resolution policy. Compare latency, availability, and returned versions under concurrent updates and network partitions. Run three replicas per key and expose R and W as client settings. Use vector clocks to distinguish concurrent writes; return conflicting versions for explicit reconciliation. Partition the cluster, update the same key on both sides, then reconnect and verify the declared resolution rule. Report stale-read frequency for different quorum settings. Concepts: quorums, version vectors, concurrent updates, consistency and availability.

3. Fault-Tolerant Chord Network

Implement Chord lookup and node membership with replicated values on successor nodes. Detect failures, repair routing information, and recover data from surviving replicas. Test lookup correctness while nodes join, leave, and fail. Use a ring of 8–12 processes with finger tables, successor lists, and periodic stabilization. Tag stored values with versions and rebuild replica copies after membership changes. Kill adjacent ring nodes and compare successful lookups, routing hops, and stabilization time against a failure-free run. Concepts: distributed hash tables, overlay routing, stabilization, replication, churn.

4. Chord with Heterogeneous Nodes

Implement a Chord-based store whose nodes have different configured storage capacities. Add capacity-aware placement through virtual nodes and redistribute values when membership changes. Measure storage balance, lookup cost, and data moved during redistribution. Assign nodes different capacity weights and give each physical node several virtual identifiers. Maintain a directory from virtual identifiers to physical processes and preserve at least one copy while transferring keys. Insert a larger node during an active workload and verify that every key remains retrievable. Concepts: overlay routing, virtual nodes, heterogeneous resources, load balancing, live migration.

5. Fault-Tolerant CAN Network

Implement a two-dimensional Content Addressable Network with key-value storage and zone-based routing. Replicate values across neighboring zones and recover ownership of a failed node's zone. Demonstrate that routing and retrieval recover after failures and subsequent node joins. Represent zones as rectangles in a unit square and forward lookups through adjacent zones. Resolve concurrent takeover attempts using a deterministic ownership rule and versioned zone records. Run 8–12 nodes, kill one, and report failed lookups, routing hops, and time to restore replica coverage. Concepts: multidimensional overlays, decentralized routing, replication, failure detection, ownership recovery.

6. Load-Balanced CAN Network

Implement a CAN store with nodes of unequal configured capacity. Add capacity-aware zone splitting and redistribution of overloaded zones while preserving routing and data ownership. Evaluate load balance and migration cost under a skewed request workload. Use storage and request-rate thresholds to trigger a zone split or transfer to a less-loaded neighbor. Keep versioned ownership records and forward requests arriving at an old owner during migration. Run uniform and hotspot workloads on 8–12 nodes and compare the busiest node's load with the average. Concepts: space partitioning, distributed load balancing, ownership transfer, data migration.

7. Resilient Tapestry Overlay

Implement a simplified Tapestry overlay for locating objects across nodes. Maintain locations for replicated objects and repair routing entries when nodes fail. Demonstrate object discovery through an alternative replica after a storage or routing node becomes unavailable. Use fixed-length identifiers and prefix-based routing tables, with periodic object-location advertisements. Expire stale advertisements and retain alternate routes to object replicas. Run 8–12 peers, remove a routing hop, and report successful lookups, routing hops, and recovery time; verify retrieved objects by content hash. Concepts: prefix routing, location indirection, soft state, replication, overlay repair.

8. Tapestry with Adaptive Object Placement

Implement a Tapestry-inspired object-location service with capacity-aware replica placement. Select among available replicas using measured or simulated latency, and update location records when objects move. Compare retrieval latency and storage balance with a fixed-placement baseline. Maintain a small candidate set per object and choose replicas using configured capacity and observed response time. Avoid moving objects on every latency fluctuation by using a threshold and cooldown. Introduce a hotspot and a slow peer, then measure placement changes, transferred bytes, and retrieval success. Concepts: replica selection, load balancing, location updates, adaptive placement.

9. Napster with a Replicated Directory

Implement Napster-style file sharing with a central directory that indexes files available on peers. Replicate directory updates across servers and provide leader failover with stale-registration cleanup. Demonstrate search and peer-to-peer download after the active directory server fails. Use three directory replicas with a consensus-backed update log, while file contents remain on peers. Peers refresh time-limited registrations; searches return candidate sources rather than transferring files through the directory. Kill the directory leader during registration and verify that committed entries survive and expired peers disappear. Concepts: metadata replication, leader election, leases, service discovery, control and data planes.

10. Napster with Reliable File Transfers

Implement a Napster-style directory and direct file transfers between peers. Divide files into verified chunks, persist download progress, and resume from alternative peers when a source disconnects. Verify the reconstructed file and measure work repeated during recovery. Use three directory replicas or one persistent directory and 5–8 file-sharing peers. Give each chunk a content hash and request identifier, and download from multiple sources with bounded concurrency. Disconnect a source halfway through a large file and report completion time, retransmitted bytes, and final hash equality. Concepts: peer-to-peer transfer, chunking, integrity verification, checkpoint recovery, idempotent retries.

11. Gnutella with Efficient Search

Implement decentralized file discovery using a Gnutella-style peer network. Add adaptive query forwarding and cached search results with expiry to reduce unnecessary messages. Compare discovery success and message overhead against fixed-TTL flooding under the same workload. Give each query a unique identifier and suppress repeated forwarding. Adjust TTL or forwarding fan-out using recent discovery success, while cached locations expire unless refreshed. Use 10–20 peers with reproducible file placements; include peer departures so stale caches are exercised, and report the tradeoff between message savings and missed files. Concepts: gossip-style dissemination, duplicate suppression, caching, bounded flooding, churn.

12. Gnutella with Improved File Availability

Implement Gnutella-style discovery and peer-to-peer file retrieval. Replicate shared files across peers and maintain or replenish replicas as peers leave. Measure file availability and discovery cost during repeated peer failures and reconnections. Track content hashes and target replica counts using periodic advertisements. Select replacement peers using free capacity and avoid starting duplicate repair transfers for the same missing copy. Use 10–20 peers, disconnect several original sources, and report available files, repair bytes, and time to restore redundancy. Concepts: decentralized discovery, replication, anti-entropy, membership, repair coordination.

13. GFS with Adaptive Replication

Implement a small GFS-inspired file system with a master and chunk servers. Adjust replication according to access load and place replicas across simulated geographical regions. Evaluate retrieval performance and availability when chunk servers or a simulated region become unavailable. Use fixed-size chunks, a persistent master mapping, and 4–6 chunk servers with simulated region labels and link delays. Collect access counters, add replicas for hot chunks, and avoid placing every copy in one region. Demonstrate a hotspot followed by a region outage and report migration bytes and recovery time. Concepts: chunk placement, replication, locality, failure domains, adaptive load balancing.

14. GFS with Reliable Append and Metadata Recovery

Implement a GFS-inspired file system with record append and durable master metadata. Use request identifiers to deduplicate retried appends, and replicate metadata to support master failover. Crash the master during an append and verify that a retried request does not create an extra record. Use three metadata replicas with consensus and 4–6 chunk servers. Assign an append request ID before transmission, persist its completion status, and define when an append is acknowledged. Inject crashes between metadata and chunk updates, then reconcile unfinished operations and compare records with the client's acknowledged-request log. Concepts: replicated metadata, deduplication, durable logging, leader failover, recovery coordination.

15. Haystack with Adaptive Fault Tolerance

Implement a Haystack-inspired store for small image files using volume-based storage and compact indexes. Add dynamic replication and recovery after storage-node failures. Verify that images remain retrievable within the configured redundancy limit and that missing copies are rebuilt. Pack image data into append-only volumes and maintain an ID-to-offset index; verify reads with checksums. Keep a persistent replica map and use access counters or failure risk to change each volume's copy count. Kill a storage node during repair and report retrieval availability, repair duration, and additional storage consumed. Concepts: object storage, replica placement, durable indexes, integrity checks, fault tolerance.

16. Haystack with Distributed Similarity Search

Implement a Haystack-inspired image store with a similarity index distributed across storage nodes. Use a fixed perceptual-hash distance to find similar images within and across collections, and merge results from different nodes. Evaluate retrieval correctness, similarity-search latency, and cross-node communication. Choose a small supplied image collection and compute perceptual hashes locally, rather than training a model. Route queries to index shards, return each shard's nearest candidates, and merge a global top-k. Replicate index partitions or rebuild them from stored images after failure; compare results with a sequential scan. Concepts: distributed indexing, scatter-gather, top-k merging, partitioning, recovery.

17. Connected Components and Minimum Spanning Tree on Ray and Erlang/OTP

Implement connected components and a minimum spanning forest using iterative label propagation and Boruvka's algorithm on both Ray and Erlang/OTP. Partition graph data across workers and implement inter-partition communication, round coordination, and termination detection. Compare correctness, runtime, communication volume, and load balance. Test increasing graph sizes and verify both algorithms against a sequential reference. Use a seeded graph generator and include skewed-degree and disconnected graphs. Exchange batched boundary updates and detect completion only when no worker has pending changes. Inject a delayed worker and measure its effect; record partitioning time separately from algorithm runtime. Concepts: graph partitioning, message passing, synchronization barriers, termination detection, communication cost.

18. Connected Components and All-Pairs Shortest Paths on Ray and Erlang/OTP

Implement connected components and all-pairs shortest paths using label propagation and batched distance relaxation on both Ray and Erlang/OTP. Partition graph data across workers and implement inter-partition communication, round coordination, and termination detection. Compare correctness, runtime, communication volume, and load balance. Use positive edge weights and initially limit shortest-path tests to 200–500 vertices. Use a seeded graph generator and include skewed-degree and disconnected graphs. Exchange batched boundary updates and detect completion only when no worker has pending changes. Inject a delayed worker and measure its effect; record partitioning time separately from algorithm runtime. Concepts: graph partitioning, message passing, synchronization barriers, termination detection, communication cost.

19. Minimum Spanning Tree and All-Pairs Shortest Paths on Ray and Erlang/OTP
Implement a minimum spanning forest and all-pairs shortest paths using Boruvka's algorithm and batched distance relaxation on both Ray and Erlang/OTP. Partition graph data across workers and implement inter-partition communication, round coordination, and termination detection. Compare correctness, runtime, communication volume, and load balance. Use positive edge weights and initially limit shortest-path tests to 200–500 vertices. Use a seeded graph generator and include skewed-degree and disconnected graphs. Exchange batched boundary updates and detect completion only when no worker has pending changes. Inject a delayed worker and measure its effect; record partitioning time separately from algorithm runtime. Concepts: graph partitioning, message passing, synchronization barriers, termination detection, communication cost.

20. Connected Components and Minimum Spanning Tree on Ray and Dask
Implement connected components and a minimum spanning forest using iterative label propagation and Boruvka's algorithm on both Ray and Dask Distributed. Partition graph data across workers and implement inter-partition communication, round coordination, and termination detection. Compare correctness, runtime, communication volume, and load balance. Test increasing graph sizes and verify both algorithms against a sequential reference. Use a seeded graph generator and include skewed-degree and disconnected graphs. Exchange batched boundary updates and detect completion only when no worker has pending changes. Inject a delayed worker and measure its effect; record partitioning time separately from algorithm runtime. Concepts: graph partitioning, message passing, synchronization barriers, termination detection, communication cost.

21. Connected Components and All-Pairs Shortest Paths on Ray and Dask
Implement connected components and all-pairs shortest paths using label propagation and batched distance relaxation on both Ray and Dask Distributed. Partition graph data across workers and implement inter-partition communication, round coordination, and termination detection. Compare correctness, runtime, communication volume, and load balance. Use positive edge weights and initially limit shortest-path tests to 200–500 vertices. Use a seeded graph generator and include skewed-degree and disconnected graphs. Exchange batched boundary updates and detect completion only when no worker has pending changes. Inject a delayed worker and measure its effect; record partitioning time separately from algorithm runtime. Concepts: graph partitioning, message passing, synchronization barriers, termination detection, communication cost.

22. Minimum Spanning Tree and All-Pairs Shortest Paths on Ray and Dask
Implement a minimum spanning forest and all-pairs shortest paths using Boruvka's algorithm and batched distance relaxation on both Ray and Dask Distributed. Partition graph data across workers and implement inter-partition communication, round coordination, and termination detection. Compare correctness, runtime, communication volume, and load balance. Use positive edge weights and initially limit shortest-path tests to 200–500 vertices. Use a seeded graph generator and include skewed-degree and disconnected graphs. Exchange batched boundary updates and detect completion only when no worker has pending changes. Inject a delayed worker and measure its effect; record partitioning time separately from algorithm runtime. Concepts: graph partitioning, message passing, synchronization barriers, termination detection, communication cost.

Additional Distributed Systems Projects

23. Distributed Mersenne Prime Search (GIMPS-inspired)

Implement a coordinator and 3–5 workers that test candidate Mersenne exponents using Lucas-Lehmer over RPC or sockets. Assign each task a time-limited lease and attempt number; workers send heartbeats and checkpoints containing the current iteration state. Reassign expired tasks from their last checkpoint and ignore duplicate or late results. Verify a reported prime with an independent worker before accepting it. Demonstrate workers joining and crashing mid-test, then report throughput versus worker count and recovery time. Use bounded candidate sizes so experiments finish on ordinary CPUs. Concepts: master-worker scheduling, leases, failure detection, checkpoint recovery, idempotency, redundant verification.

24. Peer-to-Peer Payment Ledger

Implement a permissioned payment ledger on 4–5 nodes with signed transfers, account balances, and per-account nonces. Use Raft for crash faults, or a simplified PBFT protocol with an explicit Byzantine fault limit, to agree on a transaction order. Validate signatures, nonces, and balances before applying committed transactions deterministically. Persist the log and recover returning nodes through replay or snapshots. Demonstrate conflicting transfers, a leader failure, and a network partition; compare final balance hashes and measure commit latency and recovery time. Concepts: consensus, total-order broadcast, state-machine replication, replay protection, fault models, recovery.

25. Real-Time Collaborative Text Editor

Implement a sequence CRDT such as RGA, with unique character identifiers, insertions, and deletion tombstones. Add a minimal editor and a relay that persists and forwards operations between three clients. Allow offline editing and exchange missing operations after reconnection using per-replica sequence summaries. Apply the same operations in shuffled, delayed, and duplicated orders and verify that document hashes converge. Demonstrate concurrent insertion and deletion at the same position, and report synchronization bytes and metadata growth; state when tombstones may safely be removed. Concepts: CRDTs, causality, strong eventual consistency, anti-entropy, idempotency, tombstones.

26. Distributed Federated Learning

Implement a federated-learning coordinator and 4–6 workers with private local data shards. Use a small supplied classifier and dataset; each round sends the global model, trains locally, and aggregates updates using sample-weighted FedAvg. Create both uniform and non-IID splits, with some workers seeing only a few classes. Add round identifiers and timeouts so late updates are rejected and a round can proceed with a stated minimum participation. Compare accuracy with centralized training and measure bytes per round, convergence, and behavior under slow or crashed workers. Concepts: round coordination, aggregation, stragglers, partial failures, communication cost.

27. Distributed Chat Application

Implement three chat servers, room logs, and clients that send messages with unique identifiers. Choose a per-room ordering guarantee, such as a replicated sequencer assigning visible sequence numbers, and acknowledge messages only after the required durable replicas accept them. Deduplicate client retries and let reconnecting clients request messages after their last received sequence. Recover room ownership after a server fails without reusing committed sequence numbers. Demonstrate retries, disconnection, and failover during a conversation; report delivery latency and recovery time. Concepts: ordering, sequencers, replicated logs, idempotency, delivery guarantees, session resumption.

28. Kafka-Inspired Distributed Event Log

Implement brokers storing partitioned, append-only topic logs with offsets. Producers route by key; each partition has a leader and replicated followers, with a commit watermark restricting what consumers can read. Support consumer groups, partition assignment, and durable committed offsets. Use a small consensus-backed ownership service so a failed leader is replaced without two active owners. Demonstrate leader failure, consumer restart, and rebalancing; verify that committed records survive. Report throughput and recovery time, and state at-least-once delivery and ordering within a partition. Concepts: partitioning, replication, leader election, commit points, consumer coordination, delivery semantics.

29. Redis-Inspired Distributed Cache

Implement GET, SET, DEL, and EXPIRE on sharded in-memory cache nodes. Route keys through consistent hashing with virtual nodes and keep a primary plus replica for each key. Add lazy and periodic TTL expiry and a bounded-memory eviction policy such as LRU. Detect failures, promote a replica through an agreed ownership epoch, and transfer affected keys when a node joins. State the write acknowledgement and consistency policy. Compare keys moved with modulo hashing, and measure hit rate and availability during failures. Concepts: consistent hashing, replication, expiry, failure detection, ownership, rebalancing.

30. MongoDB-Inspired Sharded Document Store

Implement JSON-document shards, each with a primary and follower, a query router, and a persistent shard map. Support insert, update, delete, key lookup, and a small filter-query API. Route shard-key queries directly and merge scatter-gather results for other queries. Replicate writes and coordinate primary failover with ownership epochs or consensus; distinguish primary reads from potentially stale follower reads. Demonstrate a primary crash and transfer one shard range while writes continue. Report query latency, moved documents, and recovery time. Concepts: sharding, replication, query routing, scatter-gather, migration, consistency.

31. BonsaiDB-Inspired Networked Document Database

Implement an embeddable document library with collections, CRUD operations, and one simple index or view. Persist updates through a write-ahead log and periodic snapshots, then expose the same operations through a network server. Replicate the log to a follower and state when a write is considered durable; implement restart recovery before accepting new requests. Compare local and remote API behavior and kill the server during an update to check acknowledged-write survival. Measure recovery time and network overhead. Concepts: durability, write-ahead logging, snapshots, log replication, client-visible consistency, crash recovery.

32. Neo4j-Inspired Distributed Graph Database

Implement a graph store partitioning vertices and outgoing edges across 3–5 nodes. Support neighbor queries, bounded k-hop traversals, and unweighted shortest-path queries through frontier exchanges between partition owners. Merge results at a coordinator and recover partitions from replicas or logs. Compare hash partitioning with one simple locality-aware strategy on a supplied graph; measure edge cuts, messages per traversal, and query latency. Disconnect a node during a traversal and retry or fail the query explicitly rather than return an incomplete result. Concepts: graph partitioning, distributed traversal, locality, replication, failure handling.

33. InfluxDB-Inspired Time-Series Database

Implement storage and query nodes for samples containing a series ID, timestamp, and value. Partition by series hash and time window, replicate partitions, and deduplicate samples by series and timestamp using a stated replacement rule. Accept late points into the correct window. Execute range queries and mean, min, max, and count through mergeable partial aggregates. Recover failed partitions from logs or replicas and demonstrate queries during recovery. Report ingestion throughput, range-query latency, and duplicate handling. Concepts: composite partitioning, idempotent ingestion, out-of-order data, scatter-gather aggregation, replication, recovery.

34. ZooKeeper-Inspired Coordination Service

Implement a 3–5-node ensemble with a small hierarchical namespace, versioned updates, and a consensus-ordered write log. Support persistent, ephemeral, and sequential nodes plus one-shot change watches. Tie ephemeral nodes to sessions and remove them after a replicated expiry decision. Build lock and leader-election recipes using sequential ephemeral nodes, with each contender watching its predecessor. Demonstrate server failure, session expiry, and reconnection; state read freshness and per-client ordering precisely. Measure operation latency and failover time. Concepts: coordination services, replicated state machines, sessions, watches, ordering, consistency guarantees.

35. Chubby-Inspired Distributed Lock Service

Implement a three-server replicated service storing locks and small files, with client sessions maintained through keepalives. Grant time-bound lock leases and a monotonically increasing fencing token on each acquisition. Build a protected storage resource that rejects requests carrying stale tokens, and preserve lock state and token counters across master failover. Add notifications for lock or file changes. Pause a lock holder past expiry, let another client acquire the lock, and verify that the old client's write is rejected. Report acquisition and failover latency. Concepts: distributed locking, leases, fencing, consensus, sessions, clock assumptions.

36. Pulsar-Inspired Distributed Messaging System

Implement messaging brokers with message storage separated from broker ownership logic. Persist topics, retention rules, and durable subscription positions. Support exclusive subscriptions with one ordered consumer and shared subscriptions with messages distributed among consumers. Acknowledge deliveries by message ID and redeliver unacknowledged messages after a timeout. Use lease-based topic ownership with fencing so another broker can safely take over after failure. Demonstrate a consumer crash and broker failover, then measure redeliveries and recovery latency. State ordering for each mode. Concepts: publish/subscribe, acknowledgements, durability, delivery semantics, ownership, recovery.

37. Elasticsearch-Inspired Distributed Search Engine

Implement sharded inverted indexes over a supplied document collection and a coordinator that merges parallel top-k query results. Use TF-IDF or BM25 with consistent corpus statistics, collected from shards or refreshed periodically. Replicate shard indexes and route queries to surviving copies after failure. Compare rankings against a single-node index and measure latency, tail latency, and throughput as shard count changes. Demonstrate one unavailable shard replica and report whether queries complete or explicitly return a partial-result status. Concepts: sharding, scatter-gather, distributed ranking, replication, shard allocation, tail latency.

38. Parallel LLM Inference

Implement pipeline inference for a small supplied language model across two worker processes on available CPUs or GPUs. Split layers between workers, transfer activations over RPC, and preserve per-request ordering and KV-cache state. Add bounded request queues, timeouts, and optional micro-batching to keep stages busy. On failure, restart the affected worker and replay the prompt to rebuild state, discarding stale outputs by request and step identifiers. Compare output with single-process inference and report token latency, throughput, and transferred bytes. Concepts: model parallelism, RPC, pipelining, backpressure, retries, recovery, communication overhead.

39. Mini Lemmy - Federated Discussion Service

Implement three independent servers hosting users and communities with posts, comments, and votes. Give each community an authoritative home server and exchange signed activities through persistent outboxes with retries and duplicate suppression. Buffer comments whose parent has not arrived, or fetch the parent. Store one versioned vote per voter and tombstones for deletions so reordered deliveries converge. Demonstrate an offline server catching up and an edit racing with deletion. Report delivery delay and queued updates. Concepts: federation, at-least-once delivery, idempotency, causal dependencies, eventual consistency, tombstones, server trust.

40. Radicle-Inspired Peer-to-Peer Code Collaboration

Implement peer discovery and Git repository replication using Git's existing object storage and transfer tools. Give peers signing identities and advertise changed refs through a seed list or gossip. Follow only selected repositories and fetch missing objects after comparing refs. Store each peer's signed refs in a separate namespace and define who may update the canonical branch, such as one designated maintainer. Demonstrate disconnected maintainers creating divergent commits and later reconciling them; measure transferred bytes and synchronization time. Concepts: peer-to-peer replication, content addressing, Merkle DAGs, anti-entropy, identity, conflict resolution.

41. Onion-Routing Network

Implement a directory of relay public keys, relay processes, a client, and a destination service. Build three-hop circuits using per-hop key negotiation and wrap payloads in one encryption layer per relay. Each relay removes its layer and forwards to the next hop; use authenticated encryption from an existing library. Detect a broken relay and rebuild the circuit, with explicit treatment of in-flight requests. Log which addresses and payload information each hop can observe, and compare latency by path length. Concepts: overlay networks, layered encryption, circuit setup, failure detection, trust assumptions, anonymity and latency.

42. Mini Mastodon - Federated Microblogging

Implement three independent servers with user profiles, posts, remote follows, and home timelines. Deliver follows and fan out new posts to follower servers through persistent retry queues and idempotent receipt. Give edits increasing author versions and retain deletion tombstones so delayed creates or edits cannot restore deleted posts. Demonstrate a server reconnecting after missed updates and a user with many remote followers. Report propagation delay, queue growth, and fan-out messages per post. Use a small shared protocol rather than full platform compatibility. Concepts: federation, fan-out, retries, ordering, eventual consistency, tombstones.

43. Distributed Warehouse Robot Traffic Control

Implement a grid simulation with 50–200 robot processes and a live view of positions, reservations, and waits. First measure collisions under independent movement, then require per-cell tokens or region-based quorums before entering shared cells. Force four robots into a waiting cycle and resolve it through a wait-for graph or ordered acquisition. Kill a token holder and recover logical ownership through agreed epochs and fencing; prevent duplicate ownership, and keep physically occupied cells blocked until cleared. Report zero protected collisions, deadlock frequency, throughput, and messages per move. Concepts: mutual exclusion, deadlocks, leases, fencing, failure handling.

44. Distributed Collaborative Pixel Canvas

Implement a 1000×1000 canvas divided into tiles across nodes, with a per-user drawing cooldown. Assign tiles through consistent hashing with virtual nodes and move only affected tiles on membership changes. Replicate updates and choose concurrent pixel winners using Lamport timestamps plus node IDs. Capture timelapse frames using a Chandy-Lamport-style snapshot on reliable FIFO channels, recording in-flight updates. Construct a causally dependent drawing example where independent shard reads produce an inconsistent frame. Report tiles moved, hotspot throughput, and snapshot overhead. Concepts: hashing, logical clocks, conflict resolution, consistent snapshots, rebalancing, rate limiting.

Project Variations

45. Distributed Image Rendering Farm

Implement a coordinator that distributes a 1024×1024 image into rendering tiles across 3–5 workers using a supplied CPU ray tracer. Support worker registration, checkpoints, reassignment, and duplicate-result handling, with persistent coordinator state. Demonstrate recovery from a worker failure and coordinator restart, and compare the assembled image with a reference render. Identify tiles by scene, frame, and coordinates; accept only the current task attempt. Store completed tile hashes and verify that resumed work does not change the final image. Report speedup, duplicate work, and recovery time on the same scene. Concepts: task scheduling, leases, checkpointing, idempotency, durable coordination, load balancing.

46. Offline Collaborative Task Board

Implement a shared board with To Do, In Progress, and Done columns using CRDTs. Support offline card creation, title edits, moves, and deletion with explicit rules for concurrent changes. Synchronize three clients after reconnection and verify convergence despite different message delivery orders. Give cards stable IDs and track causal dependencies of edits and moves. Use a relay or peer synchronization that requests only missing operations; define deletion precedence explicitly. Demonstrate two clients moving one card while a third deletes it, and report merge time and metadata growth. Concepts: CRDTs, causality, offline operation, anti-entropy, convergence, tombstones.

47. Federated Campus Question-and-Answer Network

Implement three independent campus servers whose users exchange questions, answers, and discussion subscriptions. Persist outgoing updates until acknowledged and suppress duplicates using event identifiers. Use authoritative home servers, version numbers, and deletion records to synchronize correctly after disconnection without restoring deleted content. Use HTTP between servers and a minimal local interface. Buffer answers received before their question and restrict edits to the originating author or home server. Partition one server, create and delete content elsewhere, then reconnect and report catch-up traffic and propagation delay. Concepts: federation, causal dependencies, retry queues, idempotency, eventual consistency, ownership.

48. Permissionless Ledger with Lightweight Verification

Implement a five-node cryptocurrency prototype with open node admission, signed transfers, low-difficulty proof of work, and greatest-cumulative-work chain selection. Handle competing branches and reverse and reapply transactions during chain reorganization. Add a header-only client with Merkle inclusion proofs and identifier-based gossip that requests only missing transactions or blocks; demonstrate partition recovery. Use account nonces to reject replays and recompute balances on the selected branch. Keep mining difficulty low enough for local CPU demonstrations, and report fork frequency, confirmation delay, and gossip bytes. State that inclusion proves membership in a selected block, not finality. Concepts: proof-of-work consensus, gossip, probabilistic finality, chain reorganization, Merkle proofs.

49. Distributed Event Log with Dynamic Consumer Rebalancing

Implement a Kafka-inspired partitioned, replicated event log with consumer groups and persistent offsets. Reassign partitions when consumers join, leave, or fail, and reject offset updates from expired owners. Add bounded queues and backpressure; demonstrate consumer recovery without skipping committed events. Assign each partition an ownership generation, and commit offsets only from the current generation. Bound unprocessed deliveries per consumer and resume from the last committed offset. Crash a consumer between processing and commit to show duplicate delivery, then measure rebalance time and backlog drain rate. Concepts: partitioning, replication, consumer coordination, fencing, backpressure, at-least-once delivery.

50. Distributed Cache with Capacity-Aware Rebalancing

Implement a Redis-inspired sharded cache with replicas, expiry, and bounded memory. Use configured node capacities to determine placement and migrate keys when nodes join or leave while requests continue. Test concurrent writes during migration and prevent older transferred values from replacing newer updates. Use weighted virtual nodes and explicit ownership epochs. Copy a key range, forward or capture intervening writes, and switch ownership only after the destination catches up. Add a higher-capacity node under a skewed workload and report load balance, moved keys, and request failures. Concepts: consistent hashing, heterogeneous resources, replication, migration, versioning, rebalancing.

51. Distributed Search Engine with Incremental Index Recovery

Implement an Elasticsearch-inspired search engine with sharded inverted indexes and merged results. Support document insertion, replacement, and deletion using versions and deletion records to handle delayed or repeated requests. Retain an update log so a restarted replica can catch up without rebuilding its complete index, and verify convergence after disconnection. Give each shard update a sequence number and persist the replica's applied position. Reconcile deletions as well as new documents, and merge top-k results using consistent ranking statistics. Disconnect a replica during replacements and deletions, then compare recovered results with a single-node reference. Concepts: sharding, scatter-gather, incremental replication, tombstones, log replay, convergence.

52. Peer-to-Peer Code Collaboration with Offline Issue Tracking

Implement peer discovery and replication of Git repositories using Git's existing storage and transfer mechanisms. Add offline issues and comments with signed updates and deterministic reconciliation, and allow selective repository following. Demonstrate synchronization of missing updates and concurrent issue changes after disconnected peers reconnect. Give issue events stable IDs and per-author sequence numbers; define concurrent title edits, closure, and comments explicitly. Exchange missing events alongside repository updates without transferring complete histories repeatedly. Partition two peers, edit the same issue, then verify identical issue state and report catch-up traffic. Concepts: content addressing, signatures, peer discovery, anti-entropy, causal history, conflict resolution.

53. Warehouse Robot Coordination with Priorities and Safe Recovery

Implement decentralized cell and intersection reservations for 50 simulated warehouse robots, with collision avoidance and deadlock handling. Add task priorities with aging and failure recovery that rejects stale ownership commands. Keep a failed robot's occupied cell blocked until explicitly cleared, and compare throughput and waiting times under priority and failure scenarios. Use epochs or fencing at reservation recipients so a resumed robot cannot act on expired grants. Force a circular wait, pause a high-priority robot, and show that lower-priority jobs eventually progress. Report collision count, starvation, recovery time, and messages per completed task. Concepts: mutual exclusion, deadlock handling, fairness, priority aging, failure detection, fencing.

54. Collaborative Pixel Canvas with Rebalancing and Recoverable History

Implement a 1000×1000 canvas with tile-based sharding, deterministic update ordering, and recovery of missed client updates. Migrate tiles during active drawing without losing concurrent writes. Save coordinated snapshots and an update log to restore canvas state and replay its history; demonstrate live node addition and node restart. Assign ownership epochs and per-tile update positions to prevent stale owners from accepting writes. Persist migration progress and deduplicate replayed updates. Crash a shard during transfer, recover it, and compare restored canvas hashes and timelapse frames with the logged reference history. Concepts: sharding, logical clocks, live migration, consistent snapshots, log replay, idempotency.

55. Student-Proposed Distributed Systems Project

Propose and implement your own distributed system, subject to instructor approval. Submit a clear problem statement, architecture, required resources, and a plan feasible for two students within 45 days. Identify at least three distributed systems concepts and explain how they will be implemented, rather than relying entirely on existing platform features. Define the communication protocol, consistency guarantees, and failure assumptions. Demonstrate concurrent operation across multiple processes or nodes and recovery from an injected failure. Compare against a suitable baseline and report correctness, performance, and communication overhead. Concepts: specify the distributed concepts relevant to your proposal.